



ML for Science (Introduction)

Lec. 1: Saptashwa
Bhattacharyya

Contacts

Where to find me:

Where to find me: sbhattacharyya@ung.si

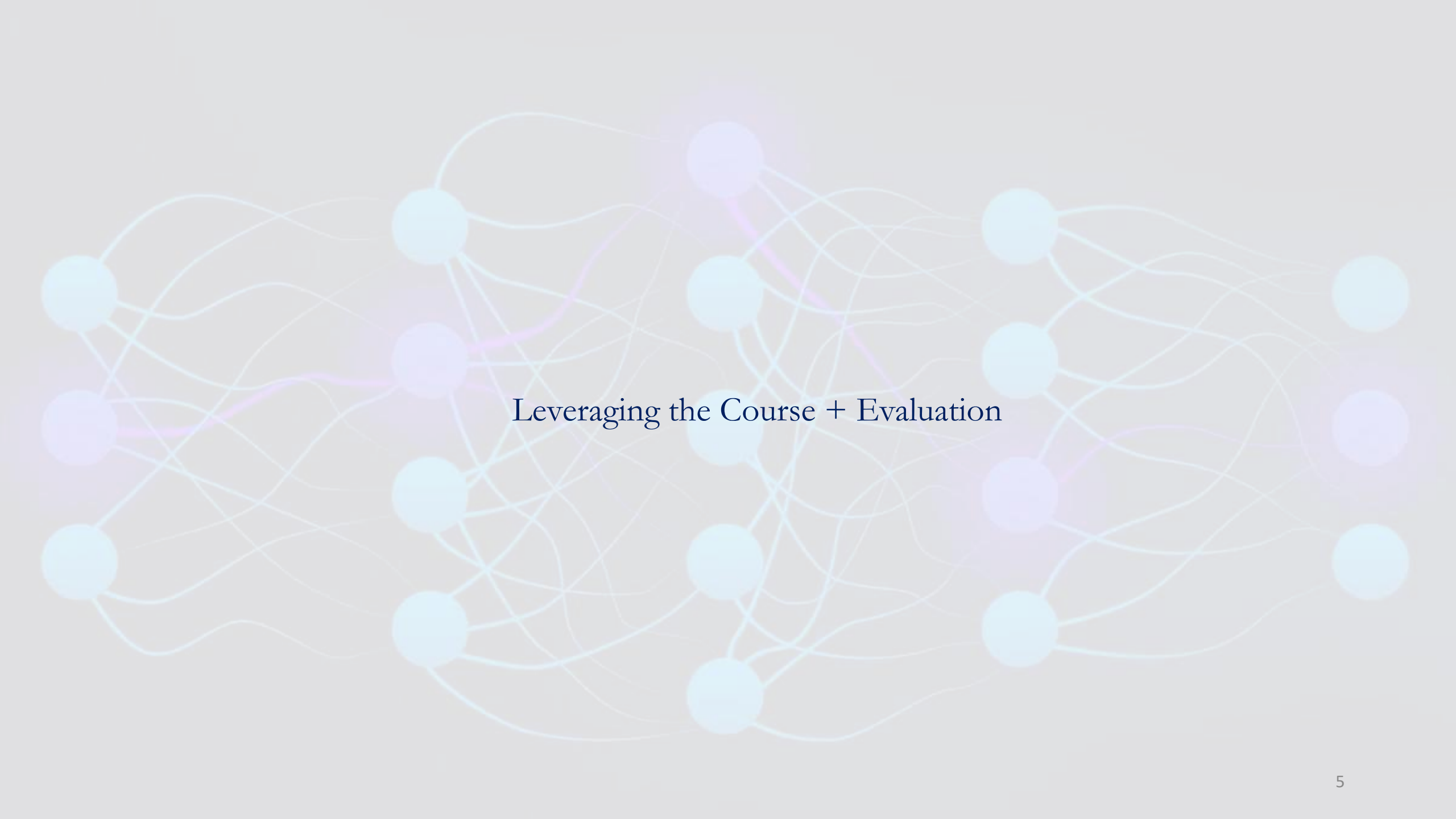


Resources:

- I will specifically guide you to necessary resources;
 - Keep an eye on the course web page (material + resources)
- If you want to get ahead and learn more:
 - Book: Concepts: “*Pattern Recognition and Machine Learning*”; C. Bishop
 - Book: Hands-on examples for basic ML (classical ML and not neural nets): “*Introduction to ML with Python*” A. Muller, S. Guido
 - Online course: Stanford Intro to ML by Andrew Ng (“the ML intro course”); Available fully for free;
- Almost all the concepts taught in this course are taught by people from MIT/Stanford; Available for free

Prerequisites

- What's mentioned on course website: None (Only Basic Python)
- What I have in mind (i.e. I think they are crucial for better understanding):
 - Linear Algebra (matrices, determinants, dot products, jacobian, ...)
 - Differential Equation (at least preliminary knowledge of function and maxima/minima)
 - Probability and Statistics (at least preliminary ideas of distributions, sampling, likelihood, ...)
 - Python as a programming language (at least ideas on arrays, dictionaries, functions, ...)
- This means more work for me!
 - But it is an opportunity to think about very common concepts in a more broad, generalized way;

A network diagram with nodes and connections, featuring a central text overlay. The diagram consists of numerous circular nodes, some of which are highlighted in a light purple color. These nodes are interconnected by a dense web of thin, light-colored lines. A few specific connections are highlighted in a darker purple color. The overall layout is horizontal, with nodes arranged in a way that suggests a flow or relationship from left to right. The background is a solid light gray.

Leveraging the Course + Evaluation

- Go beyond the example notebooks + HWs to try different examples;
 - Way too many resources + free data available on the web
- Ask me + other instructors

- Curate a GitHub Repo.
- Dedicate the GitHub repo for this course;
- Highlight additional steps that you tried, justify why they work (if they work) and why they don't (if they don't)
 - Additional step: You can start writing blogs on your findings

- Final Project: Apply the learned technique(s) on your own data (materials, chemistry, biology, physics);
- Present your findings; 25-30-minute presentations;
 - Bring your domain expertise
 - Ask for help (for ML/neural net, data handling, etc.)

- Goal: Become independent and leave with more options (apply for post-docs beyond your PhD topic, apply for jobs)
 - To help with jobs: Also add common interview question(s; 1 or 2 after hands on session)

Have Fun! (partly my job)



OR IS THE MODEL TESTING ME?

Before we Begin: Mini Motivation

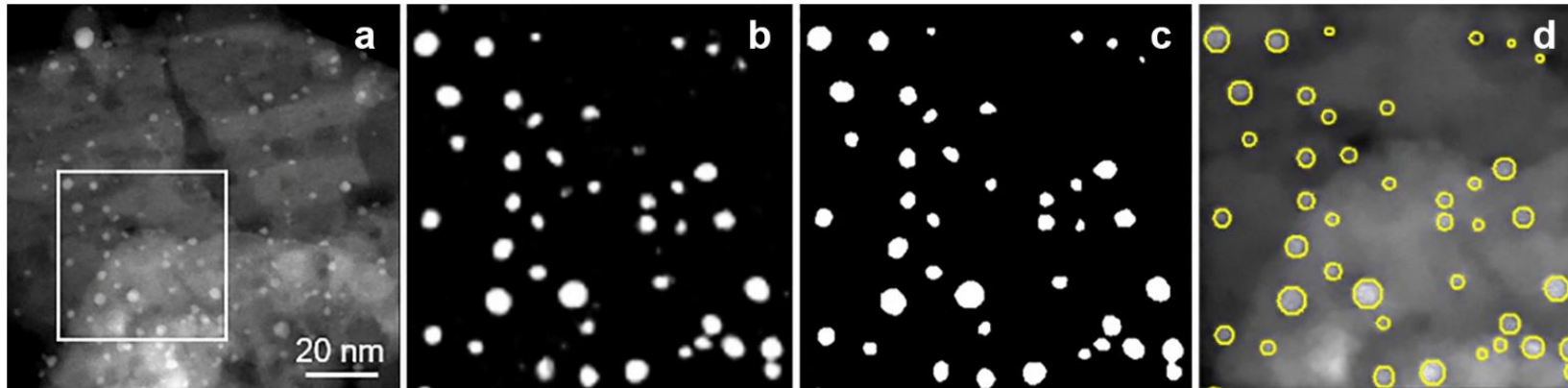
- Concepts, when learned for a specific example, can potentially be transferred into a broader class of problems;



OPEN Deep learning detection of nanoparticles and multiple object tracking of their dynamic evolution during in situ ETEM

Figure 2

From: [Deep learning detection of nanoparticles and multiple object tracking of their dynamic evolution during in situ ETEM studies](#)



From U-Net segmentation to particle detection. (a) First frame of the Pd250 series; the field of view is about $130 \times 130 \text{ nm}^2$. (b) U-Net prediction (probability map) for the enlarged white frame in (a). (c) Binary image. (d) Segmented particles represented by disks of an equal surface.

enier³, Christophe Ducottet¹ & Thierry Epicier^{2,4}✉

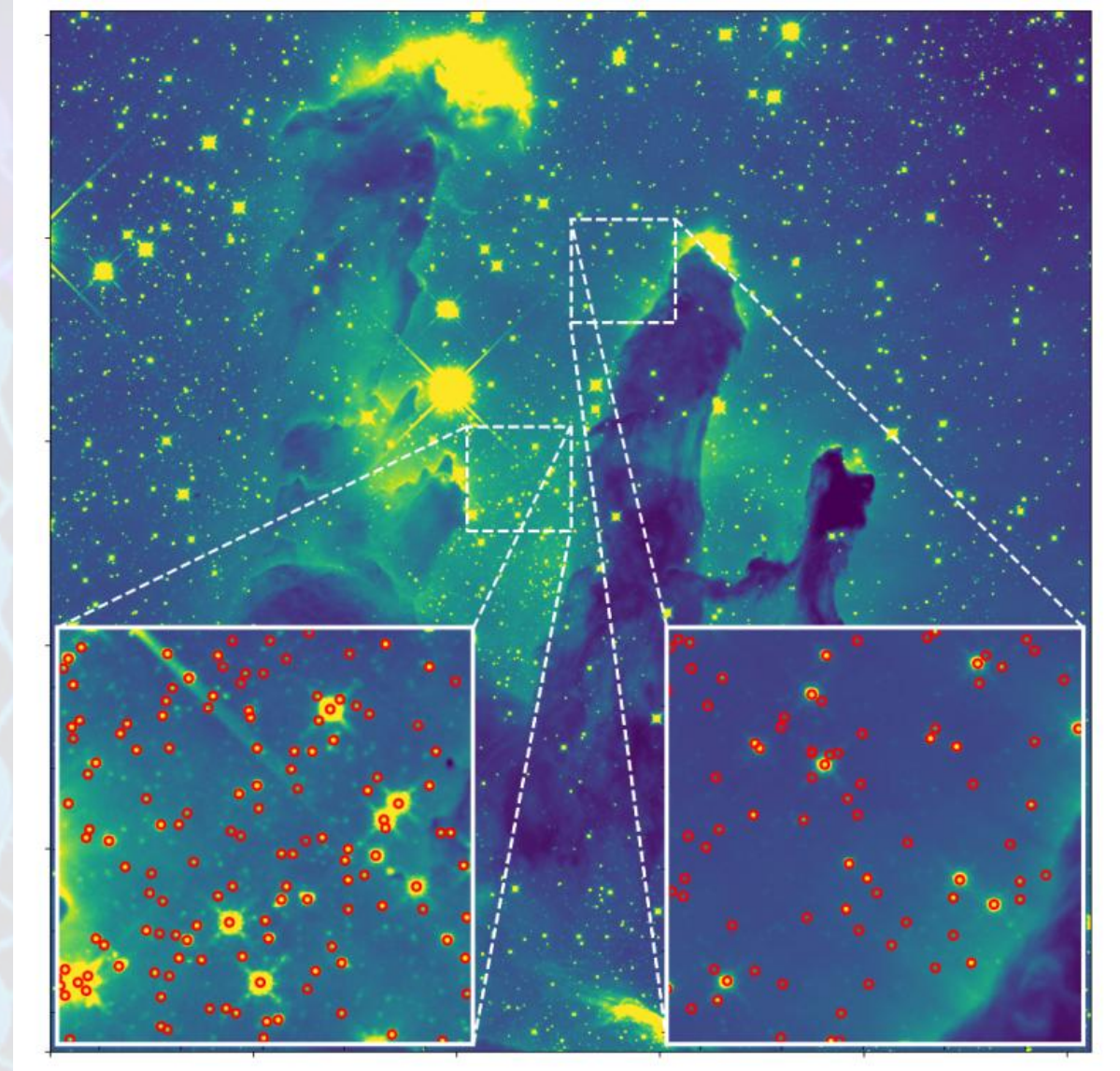
croscopy (TEM) studies of dynamic events produce large quantities of images. In the important case of heterogeneous catalysis, higher gas and temperature allows to follow a large population of nanoparticles evolving under reactive conditions. Interpreting properly large amounts of information on the catalytic properties of the active phase by machine learning is a challenge. To perform a quantitative, objective and robust treatment, a deep learning approach detects the nanoparticles on all frames of the series. A iterative tracking algorithm reconstructs their trajectories. This approach provides intuitive and statistical features about their evolution or motion, such as merging or crossing events. We treat the case of in situ calcination of palladium, where the present approach allows a discussion of operating conditions leading to NP aggregative coalescence.

Before we Begin: Mini Motivation



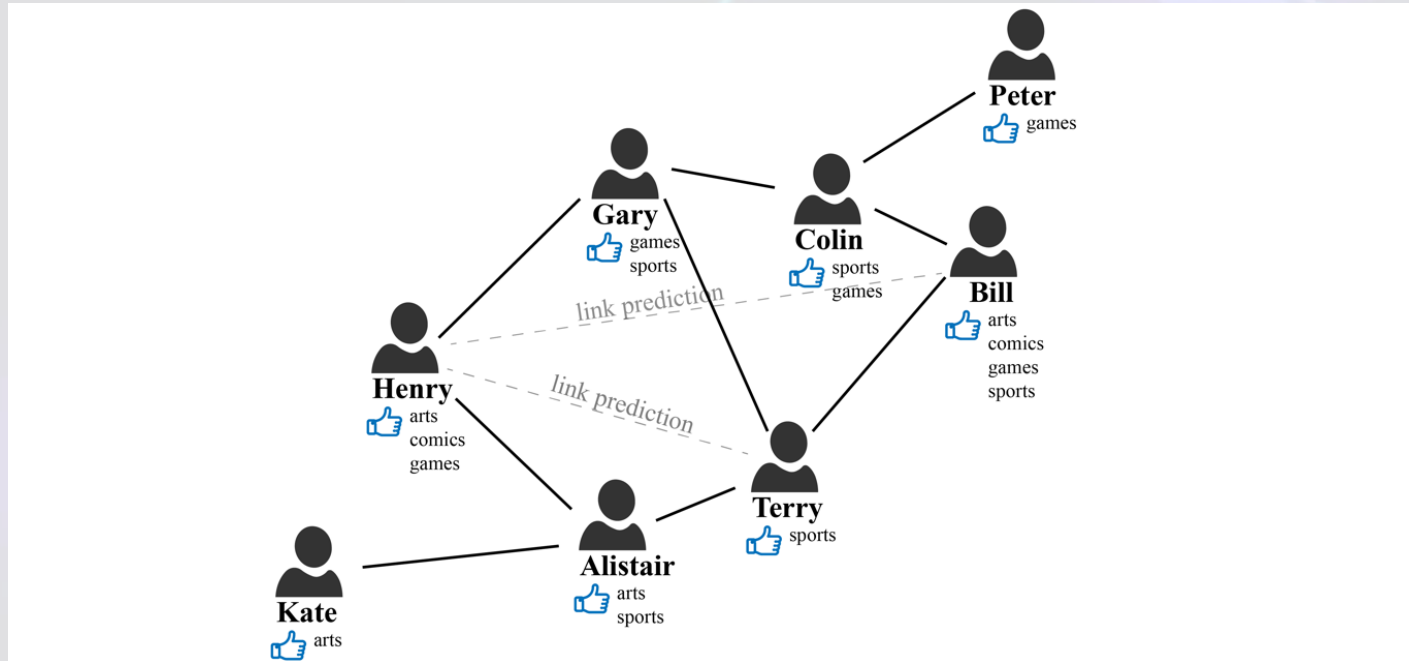
Same network, same mathematical principles, similar evaluation criterion:

- Astrophysical data
- Satellite data
- SEM/TEM data
- LIDAR data
-

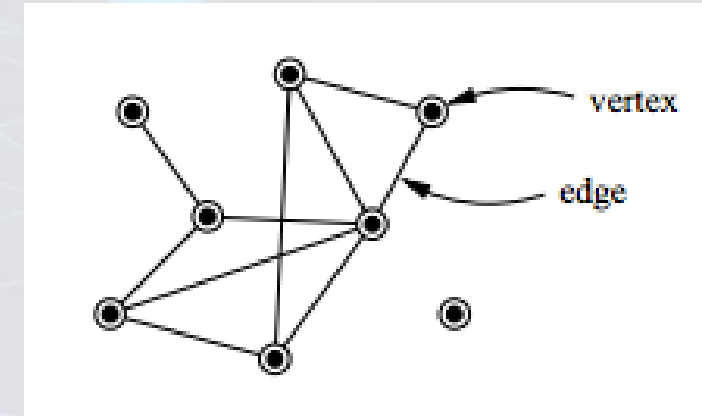


Before we Begin: Mini Motivation

Movie/Next Product Recommendation

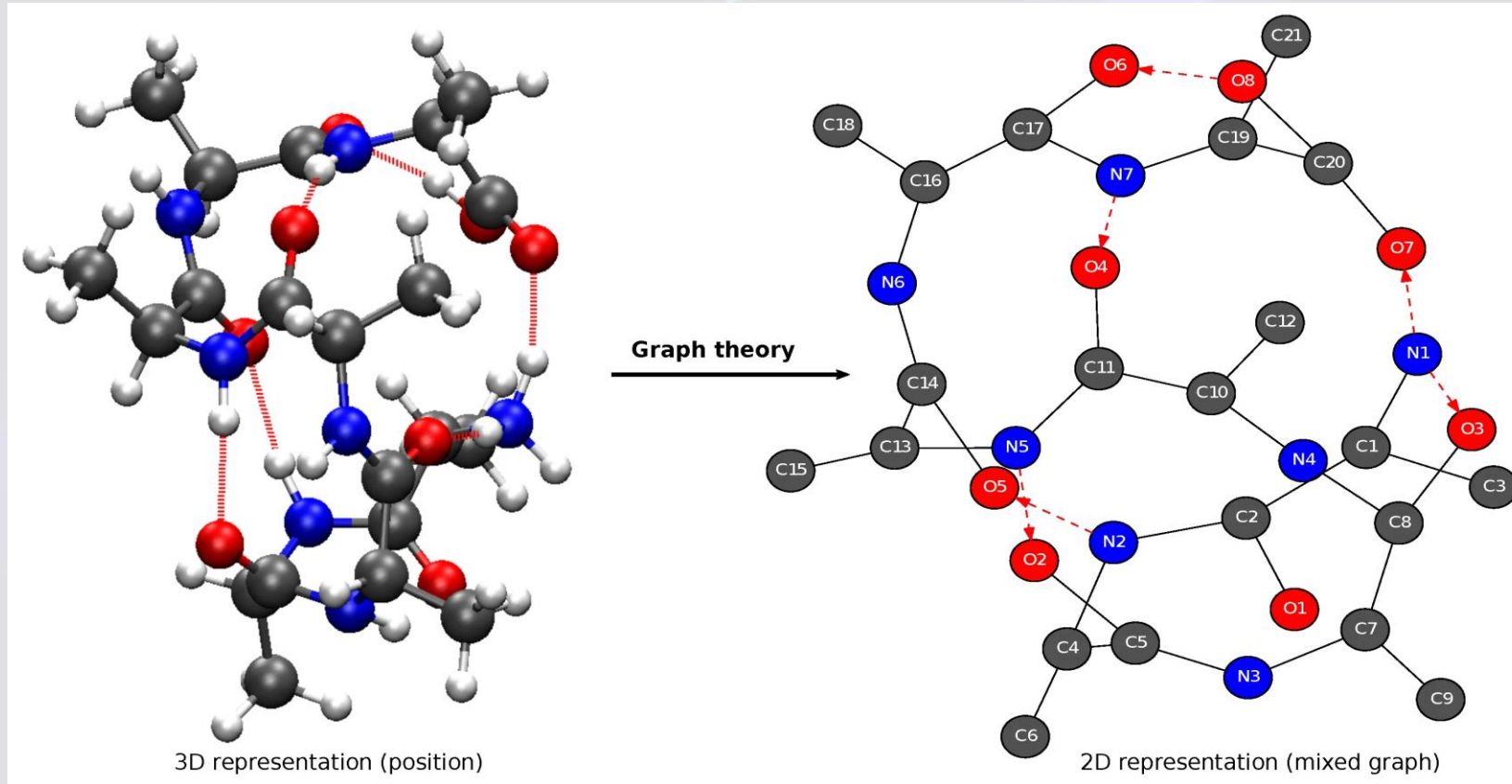


[Ref: Graph-based recommendation system: AWS Blog](#)



Simple Graph: Nodes and Edges
 $G = (V, E)$

Before we Begin: Mini Motivation



$$G = (V, E_C, E_I, E_O)$$

V : Atoms

E_C : covalent bond

E_I : Electrostatic interaction

E_O : Organometallic Interaction

[Ref: Algorithmic Graph Theory: 3D Structure to Topological 2D Molecular Graphs](#)

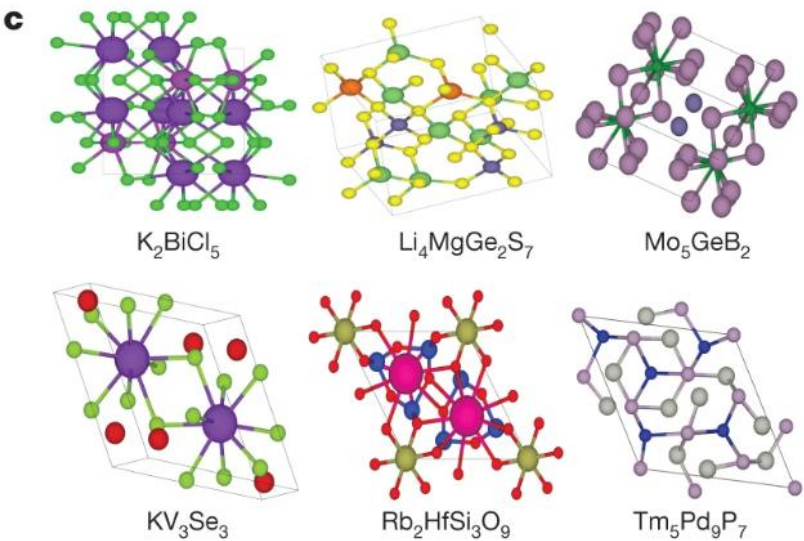
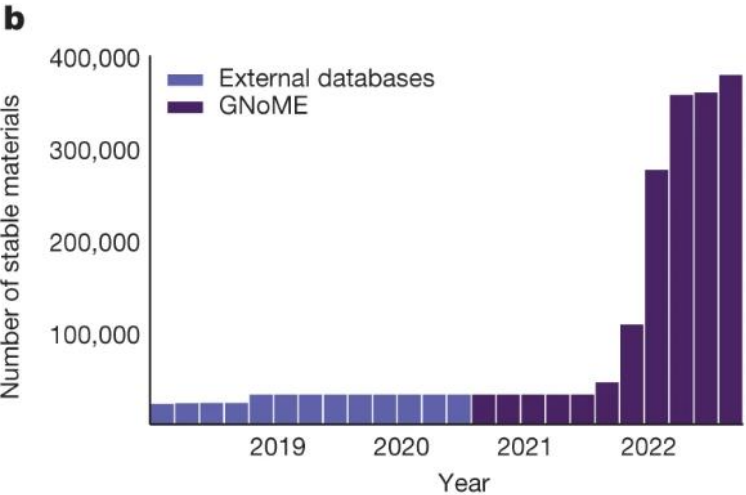
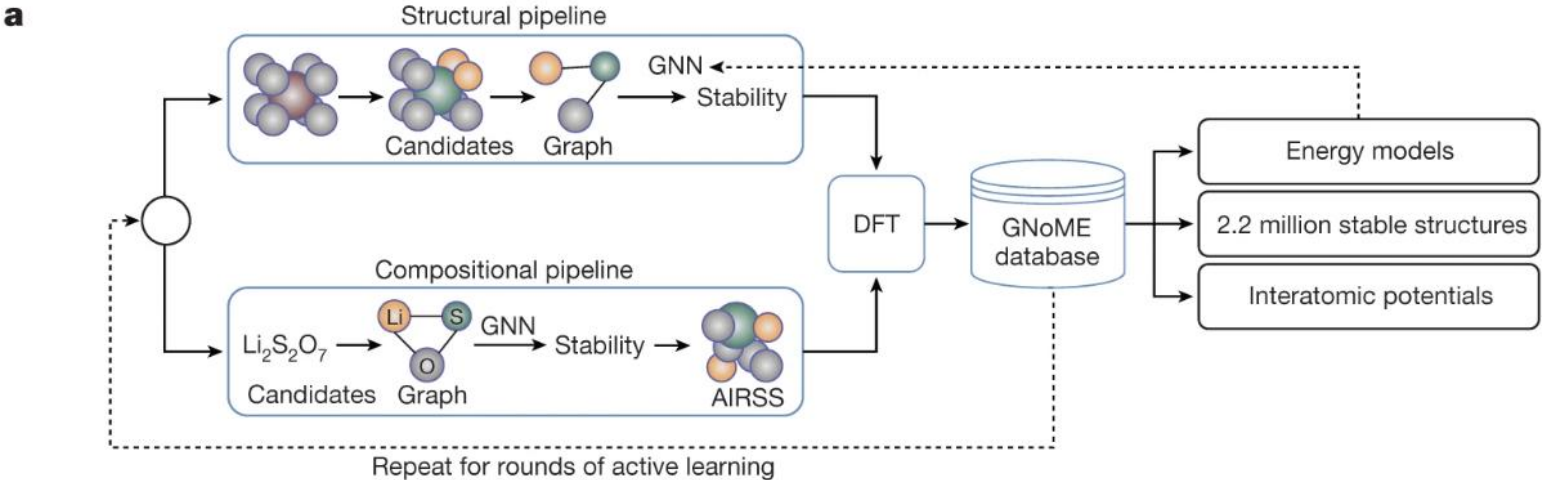
Before we Begin: Mini Motivation

Google DeepMind Models Research Science About

Millions
discovered

Amil Me

[Ref: Deepmind GNoME](#)



[Ref: Scaling Deep learning for material discovery; Nature](#)

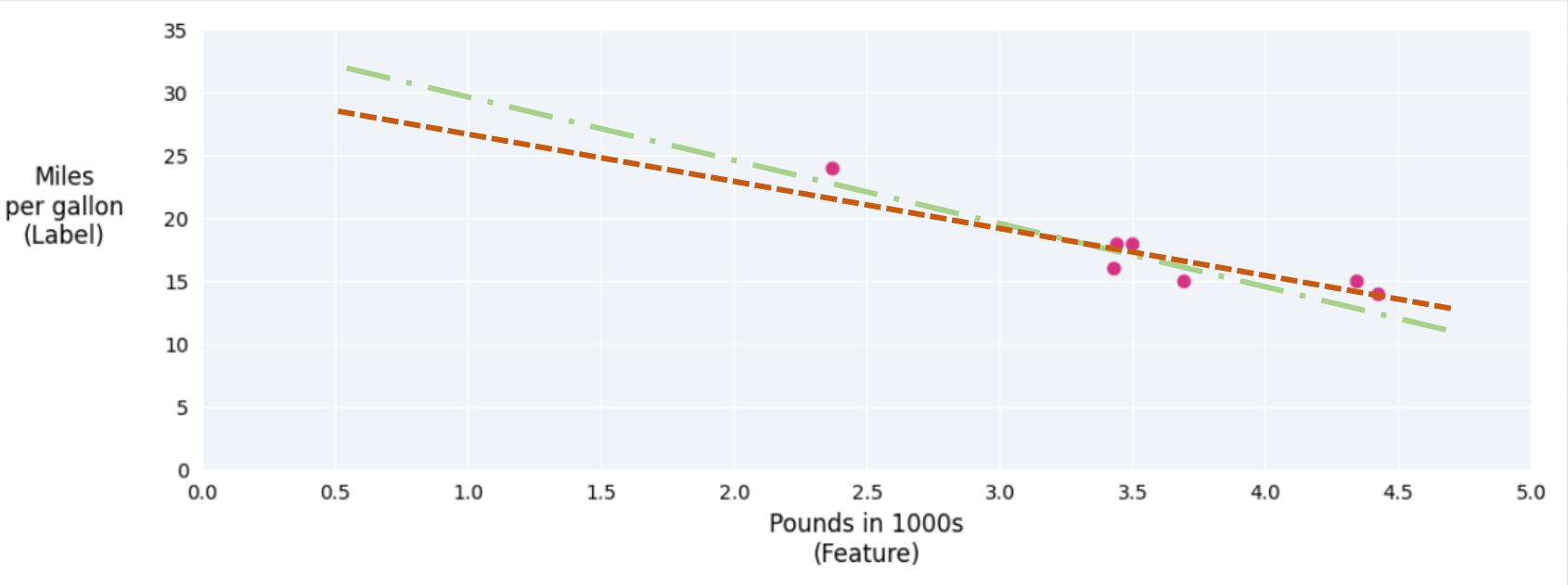


Starting from basics.....

Getting Started: Linear Regression

Car fuel efficiency given the weight of the car

Pounds ($\times 10^3$)	Miles/gallon
3.50	18
3.69	15
3.44	18
3.43	16
4.34	15
4.42	14
2.37	24



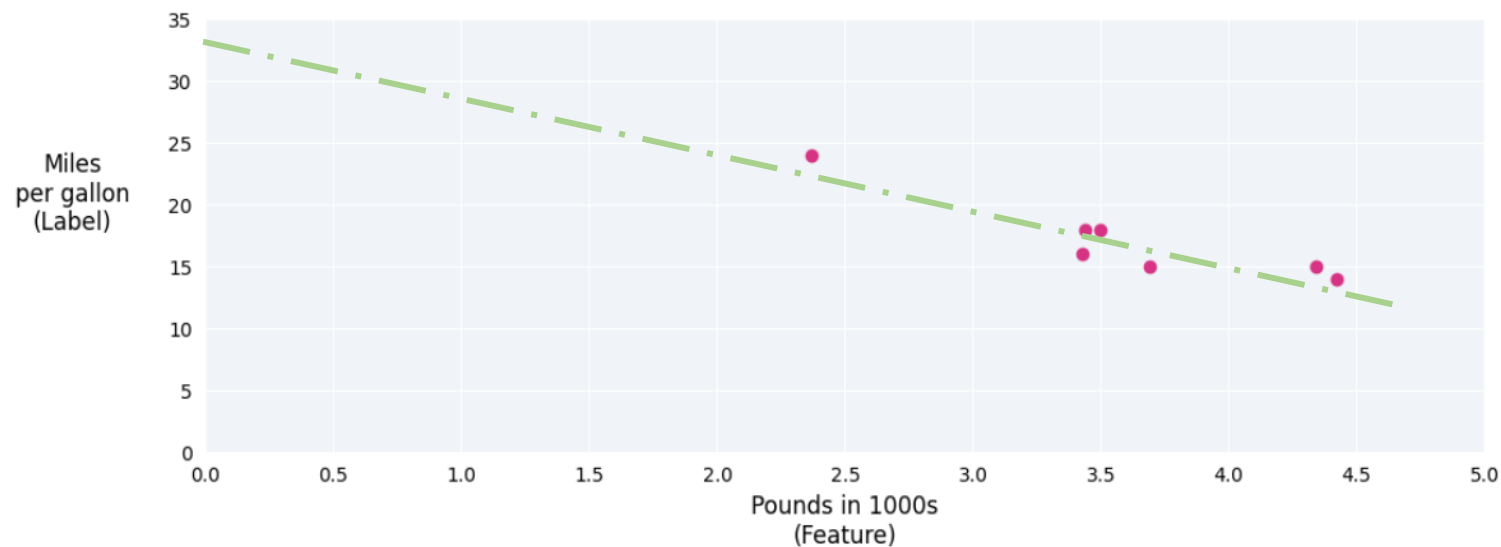
Regression == Continuous Data

Getting Started: Linear Regression: Math Notations/Concepts

- We are almost always interested in finding a functional form for input that maps to output $y \approx f(x)$
 - Math notation: $f: X \mapsto Y$
 - Single input, single output: $f: \mathbf{R} \mapsto \mathbf{R}$
 - The complete dataset: $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$
 - Fancy way of saying our dataset has N samples
- We **know** which x_i corresponds to which y_i ; The data is fed in the algorithm also in that manner
 - **Supervised Learning!!**

Getting Started: Linear Regression: Minimizing Error

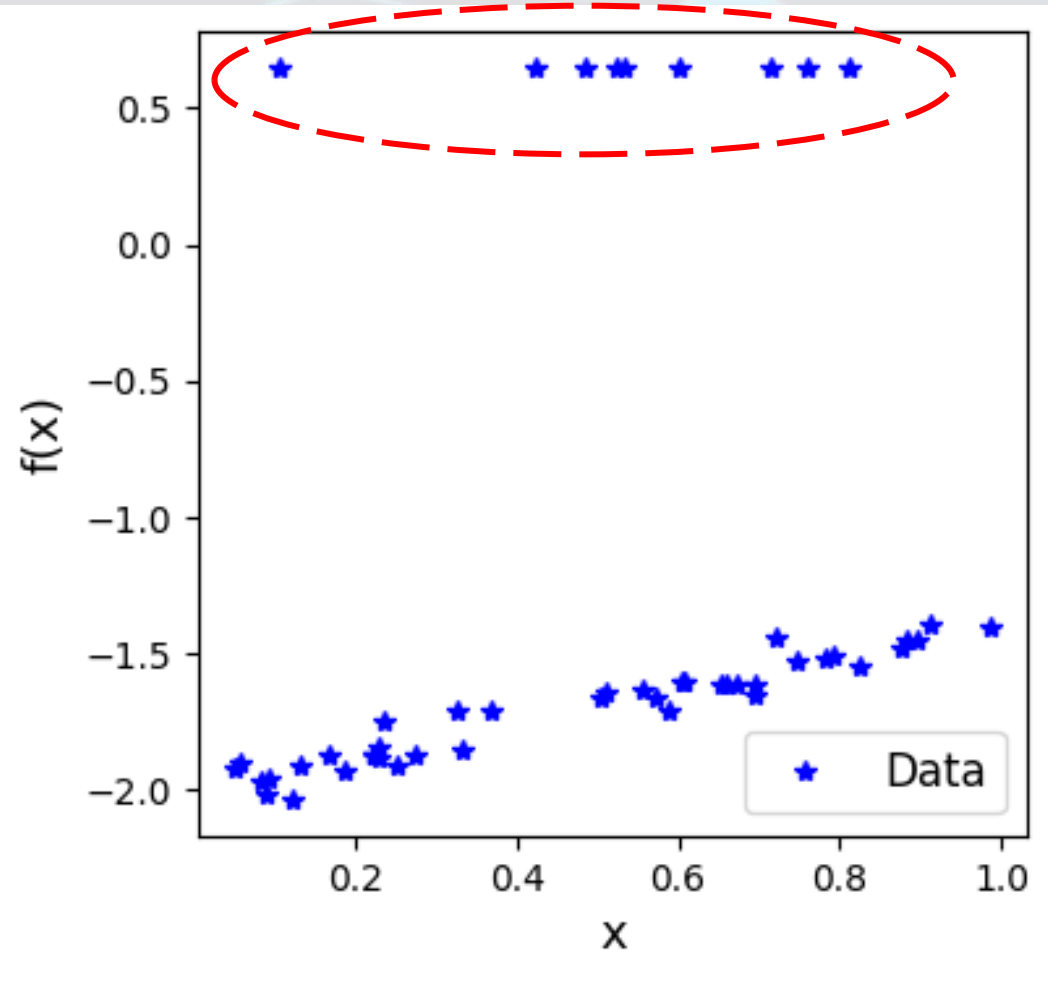
- Assumption: A linear relationship between features and labels exist: $\hat{y}_i = f(x) = wx_i + b$
 - Ground truth y_i , prediction: $\hat{y}_i$
- Calculate Mean Squared Error: Point by point: $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - (wx_i + b))^2$
- ML: here it boils down to minimizing a loss: Loss function: $\mathcal{L} = \mathcal{L}(w, b);$
 - Also introduced in some papers as: $\mathcal{L}(w, b | \mathcal{D})$



Getting Started: Linear Regression: Minimizing Error; Distance Metric

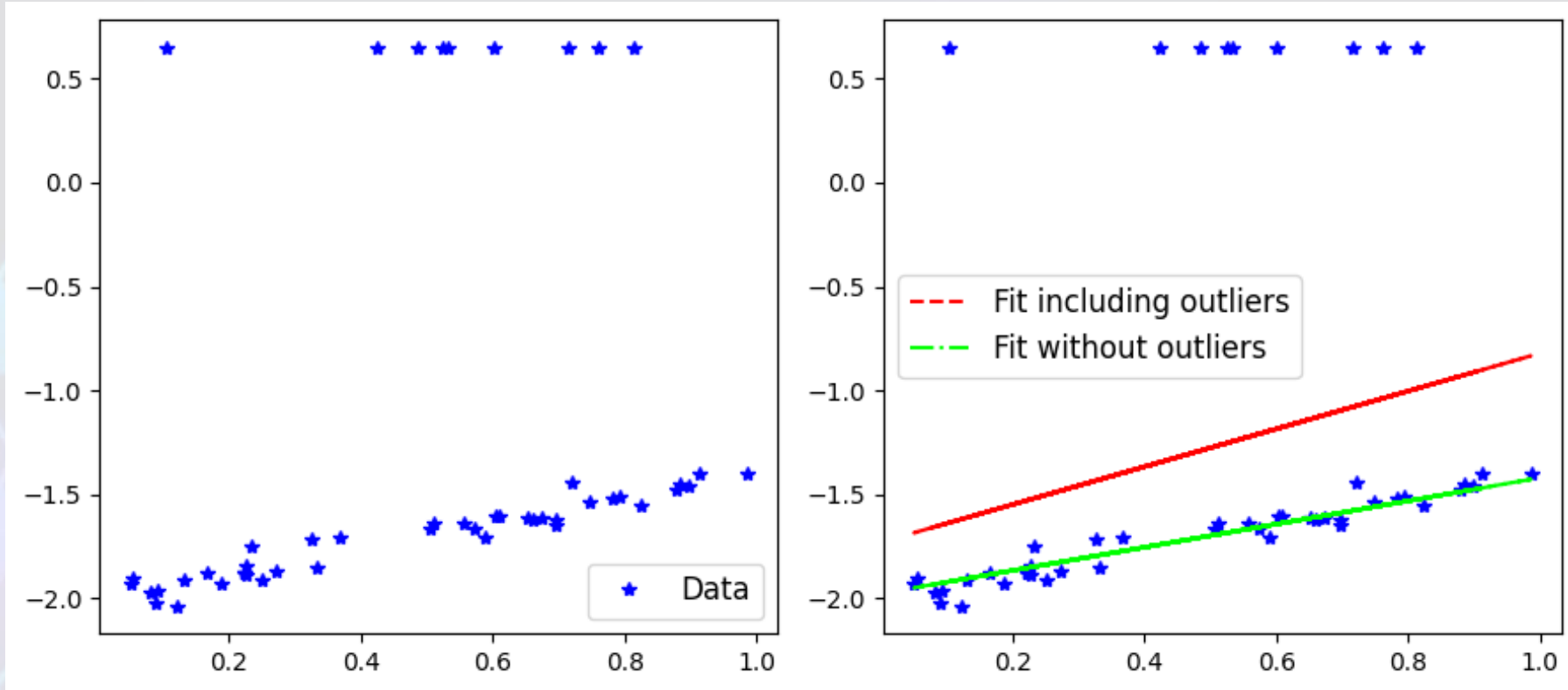
- Calculate Mean Squared Error: Point by point: $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - (wx_i + b))^2$
 - MSE is natural outcome when the noise in the data are normally distributed $y = f(x_i; w, b) + \epsilon_i; \epsilon_i \sim \mathcal{N}(0, \sigma^2)$
 - Likelihood of a single data point: $p(y_i | x_i; w, b) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f(x_i))^2}{2\sigma^2}\right)$
 - Minimizing the MSE is equivalent to maximizing the likelihood!!
- Could this be a problem?
- **Penalize larger errors (more);** Think about the tails of the Gaussian.
 - ML is not arbitrary crap!! It is quite mathematically principled!

Linear Regression: Why MSE May Not Be Good Always?



What if I know my data may contain some outliers but I don't want to drop them?

Linear Regression: Why MSE May Not Be Good Always?



$$y = f(x) = wx + b$$

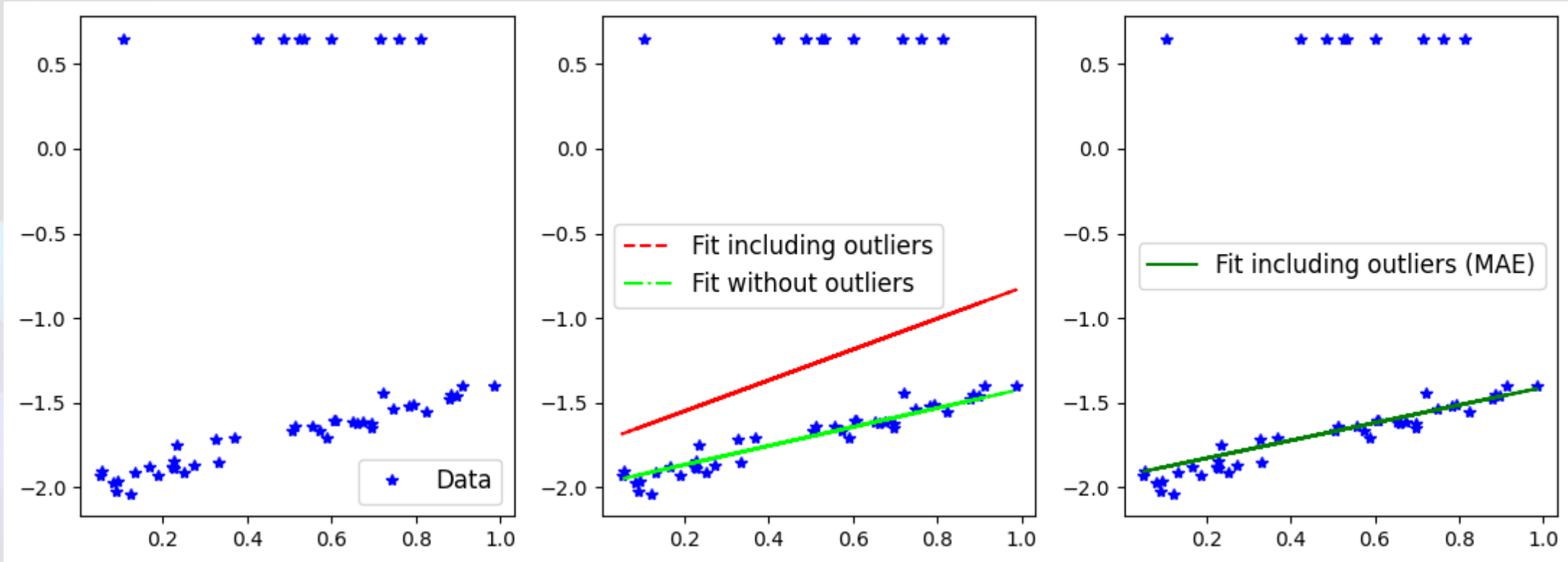
True $w = 0.6$

Predicted $\hat{w} = 0.56$
(without outliers)

Predicted $\hat{w} = 0.71$
(with outliers)

- MSE Penalizes larger errors more i.e. $(y - \hat{y})^2$; Outliers influence the fit;
- We can use MAE: Mean Absolute Error $|y - \hat{y}|$

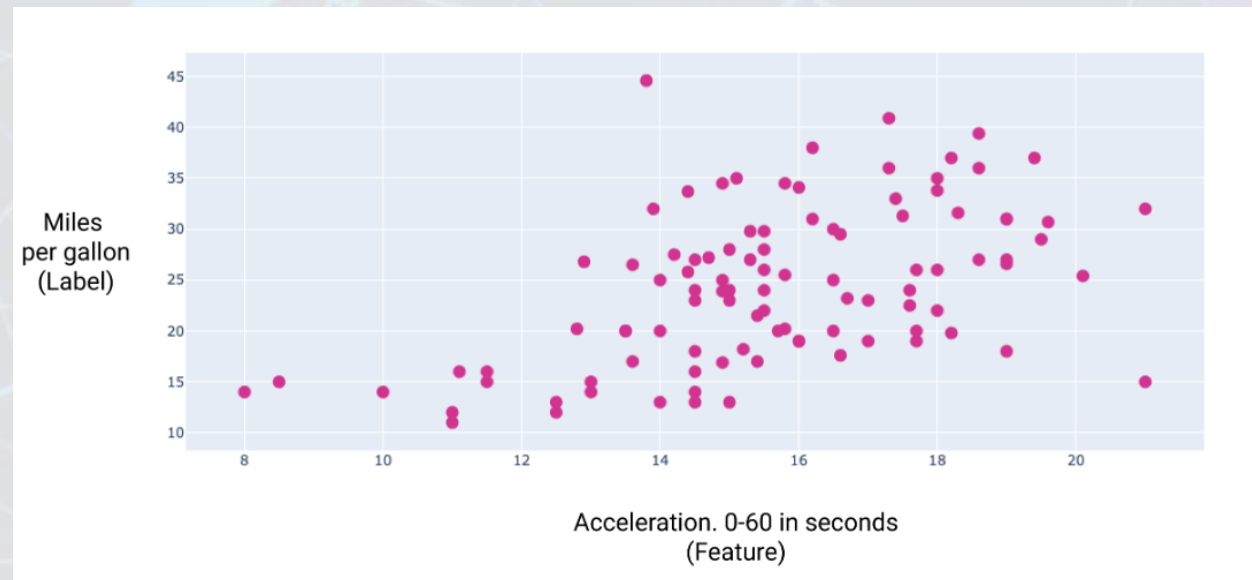
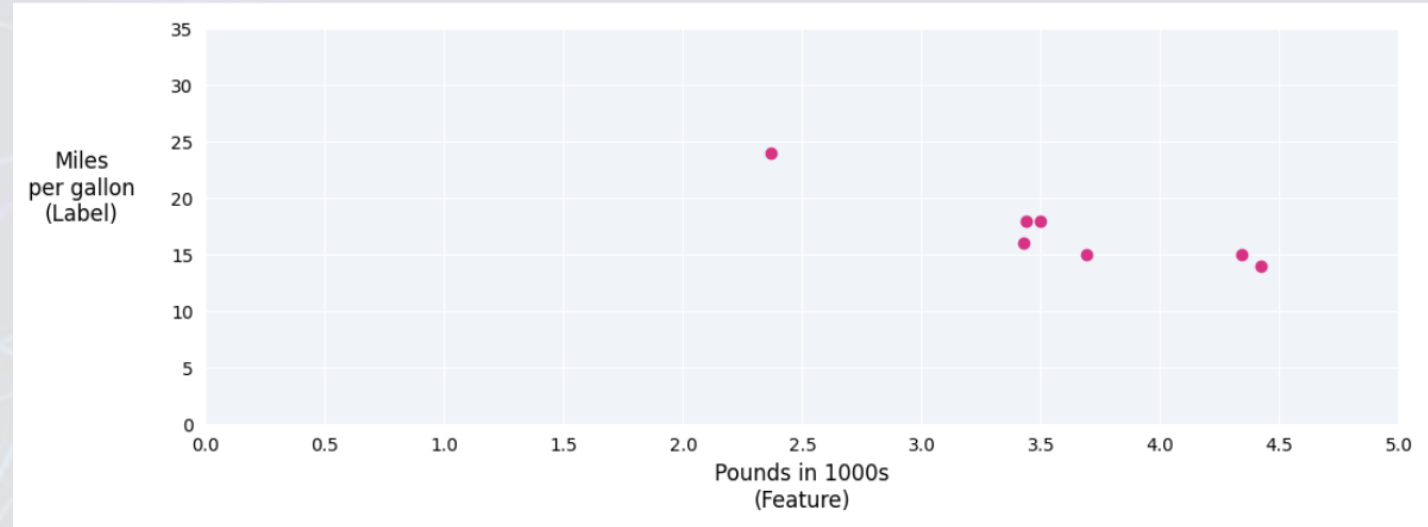
Linear Regression: MAE: Robust to Outliers



- It is quite common to test both MSE and MAE for large datasets (later on when we use Deep Neural Nets)
- Can we make a guess about the noise distribution for MAE loss? $y = f(x_i; w, b) + \epsilon_i$
- It is Laplace distribution $\epsilon \sim \text{Laplace}(0, b) \rightarrow p(\epsilon) = \frac{1}{2b} \exp\left(-\frac{|\epsilon|}{b}\right)$

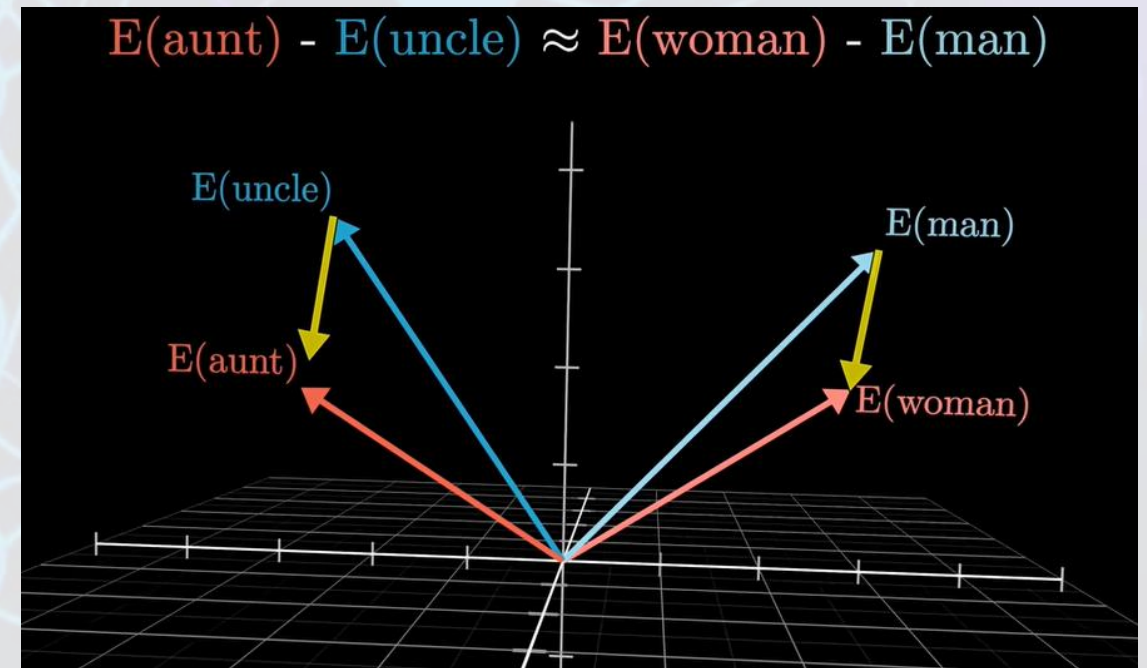
Linear Regression: Multi-Variable; Math Notations/Concepts

- So far, we discussed dataset with single feature;
 - Single input, single output: $f: \mathbf{R} \mapsto \mathbf{R}$
 - The complete dataset: $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$
- What if we have multiple features?
 - $y_i = b + w_1x_{i1} + w_2x_{i2} + w_3x_{i3} + \dots$
 - **Multi-variable regression**
- Car mileage also depends on acceleration, HP etc....



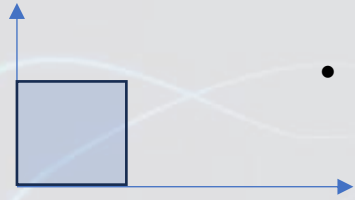
Linear Regression: Multi-Variable; Math Notations/Concepts

- Multi-variable regression: One data point is a feature vector: $\mathbf{x}_i = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$
- N number of data points, but n denotes the number of features;
 - $f: R^n \mapsto R; \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \mapsto y = f(\mathbf{x})$
- Using matrix notation we define: $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$;
 - $\mathbf{w} \in R^n; b \in R$
 - $\mathbf{w}^T \mathbf{x}$: basically, dot product of matrices;

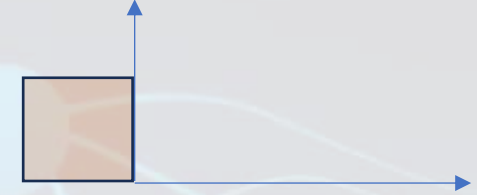


Detour: Matrices are Cool! And IMPORTANT in ML!!

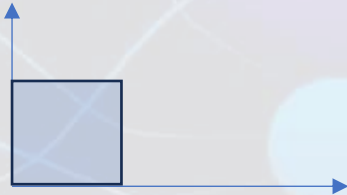
- Example of Matrix Transformations: Consider a unit square at the origin.



- Consider a matrix $A_r = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$ applied to this space



- Let's consider another matrix: $A_s = \begin{bmatrix} 1 & 1 \\ 0 & k \end{bmatrix}$; applied to the same space



$$\begin{aligned} \begin{bmatrix} 0 \\ 0 \end{bmatrix} &\rightarrow \begin{bmatrix} 0 \\ 0 \end{bmatrix}; \\ \begin{bmatrix} 1 \\ 0 \end{bmatrix} &\rightarrow \begin{bmatrix} 1 \\ 0 \end{bmatrix}; \\ \begin{bmatrix} 0 \\ 1 \end{bmatrix} &\rightarrow \begin{bmatrix} 1 \\ k \end{bmatrix}; \\ \begin{bmatrix} 1 \\ 1 \end{bmatrix} &\rightarrow \begin{bmatrix} 2 \\ k \end{bmatrix}; \end{aligned}$$

Think about your Data as a space; We apply some matrix transformation to this space

Linear Regression: Multi-Variable; Optimization

- For single variable regression, we had: $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - (wx_i + b))^2$
 - If we want to optimize for best w , we have: $\frac{\partial \mathcal{L}}{\partial w}$
- Now we have multivariable regression: $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - (w^T x_i + b))^2$
 - $\frac{\partial \mathcal{L}}{\partial w_j} = \frac{-2}{N} \sum_{i=1}^N x_{ij} (y_i - \hat{y}_i)$
 - Full loss: $\nabla_w \mathcal{L}$ (gradient of loss function $\rightarrow$ Gradient Descent Algorithm)

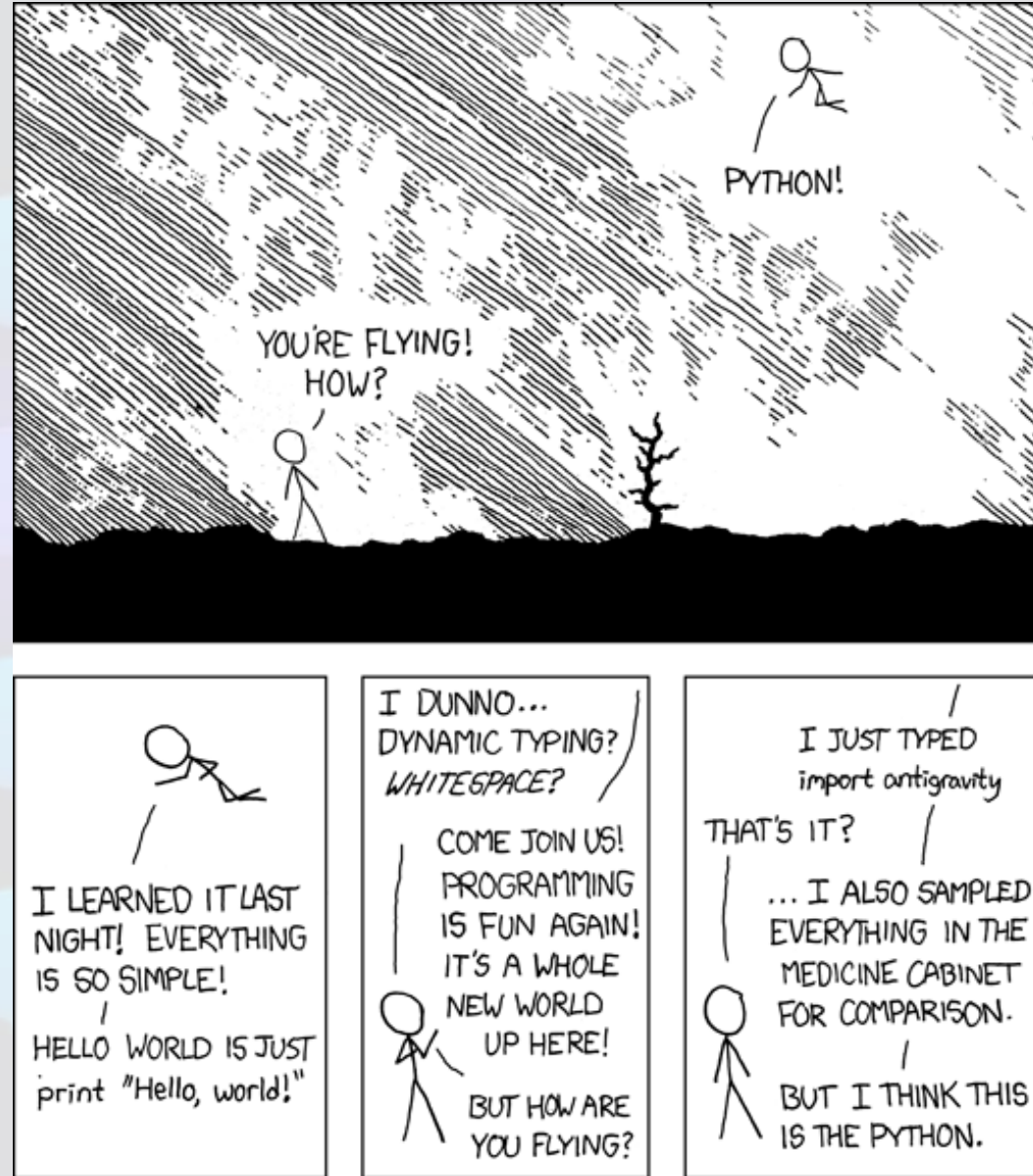
- For example, if we have only two features: $X = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \\ x_{31} & x_{32} \\ \vdots & \vdots \\ x_{N1} & x_{N2} \end{bmatrix}$; $W = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$; $y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$

- $\nabla_w \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \end{bmatrix}$

Linear Regression: Summary

- Regression works for predicting continuous variable as the outcome;
 - Dependent variable is continuous.
- Assumption: A linear relationship between input and output: $y \mapsto f(x) = wx + b = \hat{y}$
- For a dataset with N samples and n features: $X \in R^{N \times n}$, $w \in R^n$, $\hat{y} = Xw + b$
- The form of loss function is not arbitrary, depends on the nature of residual (suggestion: try both MSE, MAE)
- Essentially, learning to map from input to output by minimizing the loss (distance between true and predicted)

Python is Easy!



Python is Easy!

Load data

```
mpg_data = pd.read_csv(path_to_folder + 'auto-mpg.csv')
```

Select Feature
and label

```
X = np.log(mpg_data['weight'].values) # values turn dataframe to array  
y = mpg_data['mpg'].values
```

Create train-test split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Initiate Model
and Fit to data

```
# initialize and train the model  
model = LinearRegression()  
model.fit(X_train, y_train)
```

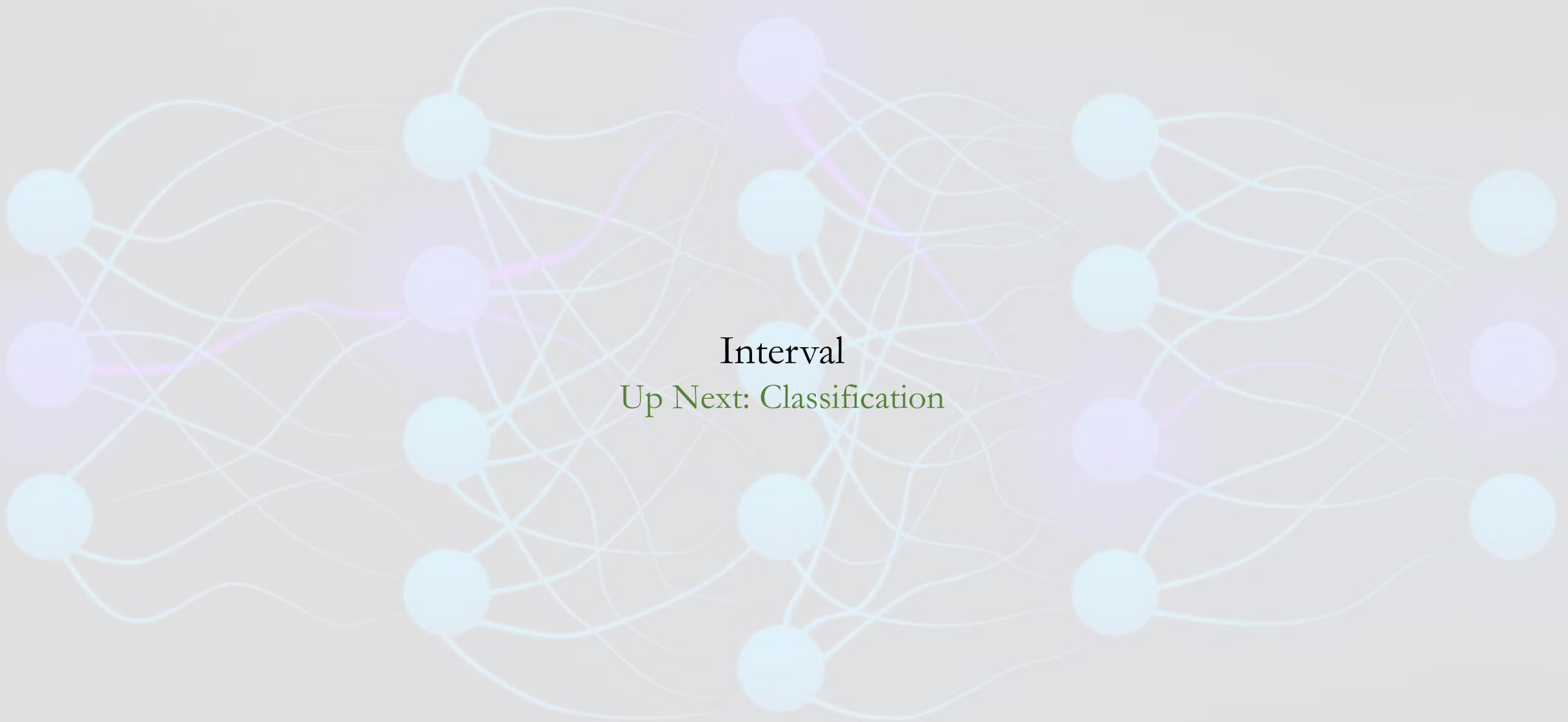
Check fit params.

```
# get back the weights (coefficients) and biases (intercept) #  $y = mx + c$  m: weights, c: biases  
weight_val = model.coef_[0]  
bias_val = model.intercept_
```

```
print(f"Model weight (slope): {weight_val:.4f}")  
print(f"Model bias (intercept): {bias_val:.4f}")
```

Predict on Test Set

```
# use this to predict on the dataset  
y_pred = model.predict(X_test)
```



Interval
Up Next: Classification